Module 8 Notes: Building Lists and Insert-Last Optimization

1. Current Progress Analysis

You are ready for Module 8 because you already understand the main ideas needed for building linked lists.

From Module 1

You learned why linked lists exist.

Arrays are fast when we already know the index, but arrays have a fixed size after creation. Linked lists are more flexible because they can grow and shrink one node at a time.

Basic tradeoff:

Structure	Strength	Weakness
Array	Fast direct access by index	Fixed size and shifting may be needed during insertion or deletion
Linked list	Flexible size and easier pointer-based insertion or deletion	No direct jumping to an index

This matters for Module 8 because building a linked list means adding nodes one by one.

From Module 2

You learned what a node is.

A singly linked-list node has two parts:

```
[data | next]
```

Part	Meaning
`data`	The value stored in the node
`next`	The address of the next node

C node definition:

```c
struct Node {
    int data;
    struct Node *next;
};
```

You also learned the important pointer movement:

```c
p = p->next;
```

This moves pointer `p` to the next node.

From Module 3

You learned traversal, display, and list creation.

Traversal means visiting nodes one by one:

```
first -> 8 -> 3 -> 7 -> 12 -> NULL
```

Basic display pattern:

```c
void display(struct Node *p) {
    while (p != NULL) {
        printf("%d ", p->data);
        p = p->next;
    }
}
```

You also learned the pointer `last` while creating a list from an array.

Important creation pattern:

```c
last->next = t;
last = t;
```

This means:

1. Attach the new node after the current last node.
2. Move `last` so it points to the new final node.

This pattern becomes very important in Module 8.

From Module 7

You learned insertion by position.

The most important insertion patterns were:

```
// Insert at the head
 t->next = first;
 first = t;
```

```
// Insert after node p
 t->next = p->next;
 p->next = t;
```

You also learned the index convention:

Index	Meaning
`0`	Insert before the first node
`1`	Insert after the first node
`2`	Insert after the second node
`n`	Insert after the last node in a list of `n` nodes

For a list with `n` nodes, valid insertion indexes are:

```
0 to n
```

So Module 8 continues from Module 7 and asks:

> Can we use insertion repeatedly to build a full list, and can we make inserting at the end faster?

---

2. Big Idea of Module 8

Module 8 teaches how to build a whole linked list by inserting nodes repeatedly.

In Module 7, you learned how to insert one node.

In Module 8, you learn how to use insertion many times to create a full list.

Example:

```
Insert(first, 0, 8);
Insert(first, 1, 3);
Insert(first, 2, 6);
```

This can create:

```
8 -> 3 -> 6 -> NULL
```

But there is a problem.

If we keep inserting at the end using the normal `Insert` function, the function may need to walk through the whole list every time.

That can become slow.

So Module 8 introduces a better method:

```
InsertLast(x)
```

`InsertLast` uses a pointer called `last` so that we can add a node at the end in constant time.

The main lesson is:

```
Normal Insert can build a list.
But InsertLast builds a list more efficiently when we are always appending at
the end.
```

---

3. Building a List Using Repeated Insert Calls

Suppose the list starts empty:

```
first = NULL
```

Now call:

```
Insert(first, 0, 8);
```

Result:

```
8 -> NULL
```

Then call:

```
Insert(first, 1, 3);
```

Result:

```
8 -> 3 -> NULL
```

Then call:

```
Insert(first, 2, 6);
```

Result:

```
8 -> 3 -> 6 -> NULL
```

Now call:

```
Insert(first, 0, 5);
```

Index `0` means insert before the first node.

Current list:

```
8 -> 3 -> 6 -> NULL
```

After insertion:

```
5 -> 8 -> 3 -> 6 -> NULL
```

Now call:

```
Insert(first, 3, 9);
```

Careful: index `3` means insert after the third node.

Current list:

```
5 -> 8 -> 3 -> 6 -> NULL
```

Count positions using the Module 7 convention:

```
Index 0: before 5
Index 1: after 5
Index 2: after 8
Index 3: after 3
Index 4: after 6
```

So inserting `9` at index `3` gives:

```
5 -> 8 -> 3 -> 9 -> 6 -> NULL
```

This shows that repeated `Insert` calls can build a full list.

---

4. Full Dry Run: Repeated Insert

Use these calls:

```
Insert(first, 0, 8);
Insert(first, 1, 3);
Insert(first, 2, 6);
Insert(first, 0, 5);
Insert(first, 3, 9);
```

Step 1

Call:

```
Insert(first, 0, 8);
```

List becomes:

```
8 -> NULL
```

Step 2

Call:

```
Insert(first, 1, 3);
```

Index `1` means insert after the first node.

List becomes:

```
8 -> 3 -> NULL
```

Step 3

Call:

```
Insert(first, 2, 6);
```

Index `2` means insert after the second node.

List becomes:

```
8 -> 3 -> 6 -> NULL
```

Step 4

Call:

```
Insert(first, 0, 5);
```

Index `0` means insert before the first node.

List becomes:

```
5 -> 8 -> 3 -> 6 -> NULL
```

Step 5

Call:

```
Insert(first, 3, 9);
```

Index `3` means insert after the third node.

The third node is `3`.

List becomes:

```
5 -> 8 -> 3 -> 9 -> 6 -> NULL
```

Final list:

```
5 -> 8 -> 3 -> 9 -> 6 -> NULL
```

---

5. Why Repeated Insert Can Be Inefficient

Repeated `Insert` works, but it may not always be efficient.

Suppose we want to build this list:

```
8 -> 3 -> 6 -> 9 -> 12 -> NULL
```

Using normal `Insert`, we might write:

```
Insert(first, 0, 8);
Insert(first, 1, 3);
Insert(first, 2, 6);
```

```
Insert(first, 3, 9);
Insert(first, 4, 12);
```

This gives the correct list.

But every time we insert at the end, the function has to move from `first` to the last node.

Example:

```
Insert 1st node: no long traversal
Insert 2nd node: maybe traverse 1 node
Insert 3rd node: maybe traverse 2 nodes
Insert 4th node: maybe traverse 3 nodes
Insert 5th node: maybe traverse 4 nodes
```

So if we build a large list by repeatedly inserting at the end without a special `last` pointer, the total work becomes large.

This can become:

```
O(n²)
```

That means the time grows very quickly as the list becomes larger.

---

6. Building at the Head

One simple way to build a list quickly is to always insert at the head.

Example:

```
Insert(first, 0, 8);
Insert(first, 0, 3);
Insert(first, 0, 6);
```

Let us dry run this.

Step 1

```
Insert(first, 0, 8);
```

Result:

```
8 -> NULL
```

Step 2

```
Insert(first, 0, 3);
```

Result:

```
3 -> 8 -> NULL
```

Step 3

```
Insert(first, 0, 6);
```

Result:

```
6 -> 3 -> 8 -> NULL
```

Head insertion is fast because it only changes two links:

```
t->next = first;
first = t;
```

So every head insertion is:

```
O(1)
```

Building `n` nodes at the head is:

```
O(n)
```

But there is one important issue.

The final order becomes reversed.

Input order:

```
8, 3, 6
```

Final list:

```
6 -> 3 -> 8 -> NULL
```

So inserting at the head is fast, but it reverses the order unless you planned for that.

---

7. Building at the End Without a Last Pointer

If we want to preserve the input order, we may insert at the end each time.

Example input:

```
8, 3, 6
```

Desired list:

```
8 -> 3 -> 6 -> NULL
```

Using repeated end insertion:

```
Insert(first, 0, 8);
Insert(first, 1, 3);
Insert(first, 2, 6);
```

This keeps the correct order.

But without a `last` pointer, each end insertion must walk to the end.

For one node, this is small.

For many nodes, it becomes slow.

If we build `n` nodes by always inserting at the end without a `last` pointer, the total time can become:

```
O(n²)
```

Why?

Because the work is like:

```
0 + 1 + 2 + 3 + ... + (n - 1)
```

That grows roughly like:

```
n²
```

So we need a better way.

---

8. The Solution: Use a Last Pointer

The better way is to keep a pointer to the last node.

We use two global pointers:

```
struct Node *first = NULL;
struct Node *last = NULL;
```

Meaning:

Pointer	Meaning
`first`	Points to the first node of the list
`last`	Points to the last node of the list

Example list:

```
first
 |
 v
8 -> 3 -> 6 -> NULL
          ^
          |
         last
```

Now if we want to add `9` at the end, we do not need to start from `first`.

We already know the last node because `last` points to it.

So we can attach the new node directly after `last`.

This makes append fast.

---

9. What Is InsertLast?

`InsertLast` is a function that always inserts a new node at the end of the list.

Instead of using an index, it only receives the value to insert:

```
InsertLast(8);
InsertLast(3);
InsertLast(9);
```

This should build:

```
8 -> 3 -> 9 -> NULL
```

The goal of `InsertLast` is:

```
Add a new node at the end in O(1) time.
```

It does this by using the `last` pointer.

---

10. InsertLast: Empty List Case

The empty list case is special.

At the beginning:

```
first = NULL
last = NULL
```

The list has no nodes.

Now call:

```
InsertLast(8);
```

We create a new node:

```
[8 | NULL]
```

Since this is the only node, it is both:

1. The first node.
2. The last node.

So we do:

```
first = last = t;
```

After this:

```
first
 |
 v
8 -> NULL
^
|
last
```

Both `first` and `last` point to the same node.

This is correct because a one-node list has the same first and last node.

---

11. InsertLast: Normal Append Case

Suppose the list already has nodes:

```
first
 |
 v
8 -> 3 -> 6 -> NULL
          ^
```

```
          |
        last
```

Now call:

```
InsertLast(9);
```

Step 1: Create the new node

```
[9 | NULL]
```

The new node has `next = NULL` because it will become the last node.

Step 2: Attach the new node after the old last node

```
last->next = t;
```

Now the list becomes:

```
8 -> 3 -> 6 -> 9 -> NULL
```

But `last` is still pointing to `6`.

Step 3: Move last to the new node

```
last = t;
```

Now:

```
first
 |
 v
8 -> 3 -> 6 -> 9 -> NULL
              ^
              |
            last
```

The core append pattern is:

```
last->next = t;
last = t;
```

This is one of the most important patterns in Module 8.

---

12. InsertLast Code in C

```c
void InsertLast(int x) {
    struct Node *t;

    t = (struct Node *)malloc(sizeof(struct Node));
    t->data = x;
    t->next = NULL;

    if (first == NULL) {
        first = last = t;
    } else {
        last->next = t;
        last = t;
    }
}
```

---

13. Explanation of InsertLast Code

Function header

```c
void InsertLast(int x)
```

This means:

`InsertLast` receives one integer value, `x`, and inserts it at the end of the linked list.

It returns nothing because it directly changes the linked list.

Create temporary pointer

```c
struct Node *t;
```

`t` is a temporary pointer used for the new node.

Important:

```
struct Node *t;
```

only creates a pointer.

It does not create a node yet.

Allocate new node

```
t = (struct Node *)malloc(sizeof(struct Node));
```

This creates a new node in heap memory.

Store the value

```
t->data = x;
```

This stores the value `x` inside the new node.

Set next to NULL

```
t->next = NULL;
```

This is important because the new node is going to become the last node.

The last node of a normal singly linked list must point to `NULL`.

Check if the list is empty

```
if (first == NULL)
```

If `first == NULL`, the list has no nodes.

So the new node must become both the first and last node.

```
first = last = t;
```

If the list is not empty

```
else {
    last->next = t;
    last = t;
}
```

This means:

1. Attach the new node after the old last node.
2. Move `last` so it points to the new last node.

---

14. Dry Run of InsertLast

Use these calls:

```
InsertLast(8);
InsertLast(3);
InsertLast(9);
```

Start

```
first = NULL
last = NULL
```

The list is empty.

Call 1

```
InsertLast(8);
```

Create new node:

```
8 -> NULL
```

Since the list is empty:

```
first = last = t;
```

Result:

```
first
 |
 v
8 -> NULL
^
|
last
```

Call 2

```
InsertLast(3);
```

Create new node:

```
3 -> NULL
```

Current list:

```
first
 |
 v
8 -> NULL
^
|
last
```

The list is not empty, so use:

```
last->next = t;
last = t;
```

Result:

```
first
 |
 v
8 -> 3 -> NULL
     ^
     |
    last
```

Call 3

```
InsertLast(9);
```

Create new node:

```
9 -> NULL
```

Current list:

```
first
 |
 v
8 -> 3 -> NULL
     ^
     |
    last
```

Attach after `last` and move `last`:

```
last->next = t;
last = t;
```

Result:

```
first
 |
 v
8 -> 3 -> 9 -> NULL
          ^
          |
         last
```

Final list:

```
8 -> 3 -> 9 -> NULL
```

---

15. Full C Program for Module 8

```c
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *next;
};

struct Node *first = NULL;
struct Node *last = NULL;

void Display(struct Node *p) {
    while (p != NULL) {
        printf("%d ", p->data);
        p = p->next;
    }
}

void InsertLast(int x) {
    struct Node *t;

    t = (struct Node *)malloc(sizeof(struct Node));
    t->data = x;
    t->next = NULL;

    if (first == NULL) {
        first = last = t;
    } else {
```

```
        last->next = t;
        last = t;
    }
}

int main() {
    InsertLast(8);
    InsertLast(3);
    InsertLast(6);
    InsertLast(9);
    InsertLast(12);

    printf("List: ");
    Display(first);

    return 0;
}
```

Expected output:

```
List: 8 3 6 9 12
```

---

16. Why Last Makes Append Faster

Without `last`, inserting at the end means:

```
Start at first
Move to second node
Move to third node
Move to fourth node
...
Reach last node
Insert
```

This takes time because we must traverse the list.

If the list has many nodes, this can take:

```
O(n)
```

With `last`, we already know the last node.

So we can directly do:

```
last->next = t;
last = t;
```

This takes:

```
O(1)
```

because the number of pointer changes is fixed.

It does not matter whether the list has 5 nodes or 5000 nodes.

The append step is still constant time.

---

17. Important Difference Between first and last

`first` and `last` do different jobs.

Pointer	Job
`first`	Gives access to the whole list from the beginning
`last`	Gives fast access to the end of the list

You need `first` to display or traverse the list.

Example:

```
Display(first);
```

You need `last` to append quickly.

Example:

```
last->next = t;
last = t;
```

Important:

```
last does not replace first.
```

If you only keep `last`, you can reach the final node, but you cannot easily display the whole list from the beginning.

If you only keep `first`, you can display the list, but you must traverse to append at the end.

Using both gives a better design for appending.

---

18. Complexity Analysis

Let `n` be the number of nodes.

Building at the head

Each head insertion changes only a fixed number of links:

```
t->next = first;
first = t;
```

So each head insertion is:

```
O(1)
```

Building `n` nodes at the head is:

```
O(n)
```

But the order becomes reversed.

Building at the end without last

If we insert at the end without a `last` pointer, we must traverse to the end each time.

Each end insertion can take:

```
O(n)
```

Building `n` nodes can take:

```
O(n²)
```

Building at the end with last

Using `last`, each append uses:

```
last->next = t;
last = t;
```

This is:

```
O(1)
```

Building `n` nodes is:

```
O(n)
```

Extra space

The extra pointer `last` uses only one pointer variable.

So the extra space for keeping `last` is:

```
O(1)
```

The list itself uses `O(n)` node space because it stores `n` nodes.

But the extra workspace for the append operation is constant.

---

19. Complexity Comparison Table

Method	Per insertion	Total time to build `n` nodes	Keeps original order?
Insert at head repeatedly	`O(1)`	`O(n)`	No, order is reversed
Insert at end without `last`	`O(n)`	`O(n²)`	Yes
Insert at end with `last`	`O(1)`	`O(n)`	Yes

Best method for appending while keeping order:

```
InsertLast with a last pointer
```

---

20. Why InsertLast Is Better Than Repeated Normal Insert at the End

Normal end insertion needs to find the last node.

That means it must walk through the list.

```
first -> node -> node -> node -> last
```

But `InsertLast` already has the last node stored in `last`.

So it skips the traversal.

Normal end insertion:

```
Find the last node first, then insert.
```

InsertLast:

```
Already know the last node, so insert directly.
```

That is why `InsertLast` is faster for repeatedly building a list at the end.

---

21. Common Beginner Mistakes

Mistake 1: Thinking `struct Node *t;` creates a node

Wrong idea:

```
struct Node *t;
```

means a node has been created.

Correct idea:

```
struct Node *t;
```

only creates a pointer.

The actual node is created here:

```
t = (struct Node *)malloc(sizeof(struct Node));
```

Mistake 2: Forgetting `t->next = NULL`

Wrong:

```
t = (struct Node *)malloc(sizeof(struct Node));
t->data = x;
```

Problem:

The new node is supposed to be the last node, but its `next` field is not set.

Correct:

```
t = (struct Node *)malloc(sizeof(struct Node));
t->data = x;
t->next = NULL;
```

The last node must point to `NULL`.

Mistake 3: Forgetting the empty list case

Wrong:

```
last->next = t;
last = t;
```

Problem:

If the list is empty, then `last == NULL`.

So this is invalid:

```
last->next
```

Correct:

```
if (first == NULL) {
    first = last = t;
} else {
    last->next = t;
    last = t;
}
```

Mistake 4: Forgetting to update `last`

Wrong:

```
last->next = t;
```

Problem:

The new node is attached, but `last` still points to the old final node.

Correct:

```
last->next = t;
last = t;
```

After appending, `last` must always point to the actual final node.

Mistake 5: Thinking `last` replaces `first`

Wrong idea:

```
If I have last, I do not need first.
```

Correct idea:

You need both.

```
first gives access to the full list.
last gives fast access to the end.
```

Mistake 6: Thinking InsertLast can insert anywhere

Wrong idea:

```
InsertLast can insert at any position.
```

Correct idea:

`InsertLast` only inserts at the end.

For position-based insertion, use the normal `Insert` function from Module 7.

Mistake 7: Thinking `InsertLast` changes the list order

Wrong idea:

```
InsertLast reverses the list.
```

Correct idea:

`InsertLast` preserves the order of insertion.

Example:

```
InsertLast(8);
InsertLast(3);
InsertLast(6);
```

Result:

```
8 -> 3 -> 6 -> NULL
```

---

22. Key Code Patterns to Memorize

Create a new node

```
struct Node *t;
t = (struct Node *)malloc(sizeof(struct Node));
t->data = x;
t->next = NULL;
```

Empty list append

```
first = last = t;
```

Normal append

```
last->next = t;
last = t;
```

Full InsertLast pattern

```
void InsertLast(int x) {
    struct Node *t;
```

```
    t = (struct Node *)malloc(sizeof(struct Node));
    t->data = x;
    t->next = NULL;

    if (first == NULL) {
        first = last = t;
    } else {
        last->next = t;
        last = t;
    }
}
```

---

23. Practice Dry Run 1

Given:

```
InsertLast(10);
InsertLast(20);
InsertLast(30);
```

Start:

```
first = NULL
last = NULL
```

After `InsertLast(10)`:

```
10 -> NULL
```

`first` and `last` both point to `10`.

After `InsertLast(20)`:

```
10 -> 20 -> NULL
```

`first` points to `10`.

`last` points to `20`.

After `InsertLast(30)`:

```
10 -> 20 -> 30 -> NULL
```

`first` points to `10`.

`last` points to `30`.

Final list:

```
10 -> 20 -> 30 -> NULL
```

---

24. Practice Dry Run 2

Given:

```
InsertLast(5);
InsertLast(8);
InsertLast(12);
InsertLast(20);
```

Final list:

```
5 -> 8 -> 12 -> 20 -> NULL
```

Pointer positions:

```
first -> 5
last  -> 20
```

---

25. Practice Questions

Question 1

What is the main purpose of `InsertLast`?

Answer

The purpose of `InsertLast` is to insert a new node at the end of the linked list efficiently.

Question 2

Why do we use a `last` pointer?

Answer

We use `last` so we can directly reach the final node without traversing from `first` every time.

Question 3

What should `t->next` be when inserting at the end?

Answer

It should be:

```
t->next = NULL;
```

because the new node becomes the last node.

Question 4

What happens during the first insertion into an empty list?

Answer

The new node becomes both the first and last node:

```
first = last = t;
```

Question 5

What are the two important lines for normal append?

Answer

```
last->next = t;
last = t;
```

Question 6

Why is appending without `last` slower?

Answer

Because we must start from `first` and traverse to the end each time.

Question 7

What is the time complexity of one `InsertLast` operation?

Answer

```
O(1)
```

because it only changes a fixed number of pointers.

Question 8

What is the total time complexity for building `n` nodes using `InsertLast`?

Answer

```
O(n)
```

because each insertion is `O(1)`, and we do it `n` times.

Question 9

What is the total time complexity for building `n` nodes by inserting at the end without `last`?

Answer

```
O(n²)
```

because each insertion may require traversal to the end.

Question 10

Does inserting at the head preserve the original order?

Answer

No.

Inserting at the head repeatedly reverses the order.

Example:

```
Insert(first, 0, 8);
Insert(first, 0, 3);
Insert(first, 0, 6);
```

Result:

```
6 -> 3 -> 8 -> NULL
```

---

26. Mini Checklist for Module 8 Mastery

You understand Module 8 if you can do all of these:

- Explain how repeated `Insert` can build a full list.
- Explain why inserting at the head is fast but reverses order.
- Explain why inserting at the end without `last` can become slow.
- Explain the role of `first`.
- Explain the role of `last`.
- Write `InsertLast` from memory.
- Dry run the empty list case.
- Dry run the normal append case.
- Explain why `t->next = NULL` is needed.

- Explain why `last = t` is needed.
- State the complexity of head insertion, end insertion without `last`, and end insertion with `last`.

---

27. Final Module 8 Summary

Module 8 teaches how to build linked lists efficiently.

In Module 7, you learned how to insert one node by position.

In Module 8, you learn how to build a whole list using repeated insertions.

The normal `Insert` function can build a list, but repeated insertion at the end can be slow because it may traverse the list every time.

A faster method is to use:

```
InsertLast(x)
```

This function uses a `last` pointer.

Important pointers:

Pointer	Meaning
`first`	Points to the first node
`last`	Points to the last node
`t`	Points to the new node

Most important empty-list line:

```
first = last = t;
```

Most important normal append lines:

```
last->next = t;
last = t;
```

Complexity summary:

Method	Total time to build `n` nodes
Insert at head repeatedly	`O(n)`
Insert at end without `last`	`O(n²)`
Insert at end with `last`	`O(n)`

The most important lesson:

```
A last pointer removes the need to traverse to the end every time.
That makes appending O(1).
```

Once you understand `InsertLast`, you understand how linked lists can be built efficiently while preserving the order of insertion.